

TRICKY

SQL QUESTIONS

All 26 questions from the interactive trainer — in one PDF

JOIN Multiplication · NULL Traps · GROUP BY · Window Functions · ORDER BY · UNION ·
UPDATE · More

Every
Trap Explained

- ◆ Exact row counts for every JOIN type — with multiplication formula
 - ◆ NULL traps that return 0 rows when you expect results
 - ◆ GROUP BY / HAVING / WHERE execution order surprises
 - ◆ RANK vs DENSE_RANK vs ROW_NUMBER with actual numbers
- ◆ UPDATE, DELETE, UNION, LIKE, BETWEEN, integer division gotchas
 - ◆ Every question: input table → surprise → why → rule → fix

INTERVIEW EDITION · 2025–2026

HOW TO USE

Cover the answer.

Guess first. Then read why.

■ WHY THIS HAPPENS

These questions are designed to SURPRISE you. Your first instinct will often be wrong. That is the whole point. The moment of surprise is when the learning sticks. For each question: look at the tables, read the query, write your answer on paper — THEN read the explanation. If you get it wrong, read the WHY box until you truly understand, not just memorise.

SECTION 1

JOIN Multiplication

Row count questions

The hardest type — matching rows MULTIPLY, they do not add

■ THE RULE — memorize this

THE FORMULA: Group all values. For each value: output rows = T1_count × T2_count. Handle NULL (always 0 rows in INNER JOIN). Handle unmatched values (excluded in INNER JOIN). Add unmatched rows back for LEFT/RIGHT/FULL JOIN. Sum everything up.

1

T1(1,1,1,2,NULL) INNER JOIN T2(1,1,1,2,4) — how many rows?

Topic: JOIN Row Count

◆ MEDIUM

Table 1 (T1)

id
1
1
1
2
NULL

Table 2 (T2)

id
1
1
1
2
4

Query: **SELECT * FROM T1 INNER JOIN T2 ON T1.id = T2.id;**

Most people guess: 4 or 5. What is the actual answer?

■ SURPRISE — Most people get this wrong!

The answer is 10, not 4 or 5. The 3 ones in T1 each pair with ALL 3 ones in T2 = 9 rows alone!

Value 1: 3 rows in T1 × 3 rows in T2 = 9 output rows

Value 2: 1 row in T1 × 1 row in T2 = 1 output row

NULL: NULL never matches anything = 0 rows

Value 4: no match in T1 = 0 rows

INNER JOIN TOTAL = 10 rows

The actual 10 output rows:

T1.id	T2.id	Which pair
1	1	T1 row1 × T2 row1
1	1	T1 row1 × T2 row2
1	1	T1 row1 × T2 row3
1	1	T1 row2 × T2 row1
1	1	T1 row2 × T2 row2

T1

T2

id	id
1	1
2	1
3	2
	2
	3
	3

Value 1: 1 (T1) × 2 (T2) = 2 rows

Value 2: 1 (T1) × 2 (T2) = 2 rows

Value 3: 1 (T1) × 2 (T2) = 2 rows

[illegible]

TOTAL = 6 rows

■ ANSWER

6 rows. Each T1 value matches exactly 2 T2 rows. $1 \times 2 + 1 \times 2 + 1 \times 2 = 6$.

4

CROSS JOIN T1(A,B,C) with T2(X,Y) — no ON condition. How many rows?

Topic: CROSS JOIN

● EASY

T1

T2

val	val
A	X
B	Y
C	

Query: **SELECT * FROM T1 CROSS JOIN T2;**

■ ANSWER

6 rows — every T1 row paired with every T2 row. $3 \times 2 = 6$.

T1.val	T2.val
A	X
A	Y
B	X
B	Y
C	X
C	Y

■ THE RULE — memorize this

CROSS JOIN = no ON condition = every row in T1 $\times$ every row in T2. Very dangerous on large tables: 1000 rows $\times$ 1000 rows = 1,000,000 rows!

SECTION 2

NULL Traps

The most dangerous SQL landmines

NULL means UNKNOWN — not zero, not empty string

■ THE RULE — memorize this

NULL compared to ANYTHING = NULL (unknown). NULL = NULL is NULL, not TRUE. NOT IN with any NULL in subquery returns 0 rows. COUNT(*) counts NULLs. COUNT(col) skips them.

5

NOT IN with a NULL in the subquery — what is returned?

Topic: NULL + NOT IN

▲ HARD

Students

Banned

id	name	student_id
1	Alice	2
2	Bob	NULL
3	Carol	

```
SELECT name FROM Students
WHERE id NOT IN (SELECT student_id FROM Banned);
```

Expected: Alice, Carol. Actual result?

■ SURPRISE — Most people get this wrong!

0 rows — completely empty! The NULL in Banned poisons the entire NOT IN.

1	Subquery returns	(2, NULL) — Banned has values 2 and NULL.
2	NOT IN expands to	id != 2 AND id != NULL for each outer row.
3	id != NULL	Evaluates to NULL (unknown), not FALSE.
4	Any AND with NULL	= NULL. Every row evaluates to NULL = not included. ALL rows excluded.

■ THE FIX

Use NOT EXISTS instead — it is always NULL-safe: SELECT name FROM Students s WHERE NOT EXISTS (SELECT 1 FROM Banned b WHERE b.student_id = s.id); -- Returns Alice and Carol correctly

■ THE RULE — memorize this

NEVER use NOT IN if the subquery column can be NULL. Always use NOT EXISTS or LEFT JOIN + IS NULL.

6

NULL = NULL — True or False?

Topic: NULL Equality

● EASY

id	name	email
1	Alice	alice@test.com
2	Bob	NULL
3	Carol	NULL

```
SELECT * FROM Users WHERE email = NULL; -- Query A
SELECT * FROM Users WHERE email IS NULL; -- Query B
SELECT * FROM Users WHERE email = email; -- Query C
```

Which queries return Bob and Carol?■ **SURPRISE — Most people get this wrong!**

Query A = 0 rows. Query C = only Alice! Only Query B returns Bob and Carol.

Query	Returns	Why
A: email = NULL	0 rows	= NULL always gives NULL, never TRUE
B: email IS NULL	Bob, Carol	IS NULL is the CORRECT way to check NULL
C: email = email	Alice only	NULL = NULL = NULL (not TRUE). Bob/Carol excluded

■ **THE RULE — memorize this**

ALWAYS use IS NULL and IS NOT NULL. NEVER use = NULL or != NULL. They always return NULL.

7

COUNT(*) vs COUNT(col) — same table, different results?

Topic: NULL + COUNT

◆ MEDIUM

id	amount
1	100
2	NULL
3	200
4	NULL
5	300

```
SELECT COUNT(*), COUNT(amount), SUM(amount), AVG(amount) FROM Orders;
```

What are the four values?■ **ANSWER**

COUNT(*) = 5, COUNT(amount) = 3, SUM(amount) = 600, AVG(amount) = 200

COUNT(*)	COUNT(amount)	SUM(amount)	AVG(amount)
5	3	600	200

■ **WHY THIS HAPPENS**

COUNT(*) counts ALL rows including NULLs = 5. COUNT(amount) counts only non-NULL values = 3. SUM ignores NULLs: 100+200+300 = 600. AVG ignores NULLs: 600÷3 = 200 (NOT 600÷5 = 120!). AVG divides by the non-NULL count, not total row count.

■ INTERVIEW GOTCHA

Interview trick: 'AVG(amount) with NULLs does NOT give you the average across all rows. It ignores NULL rows. If you want NULLs treated as 0: AVG(COALESCE(amount, 0)). Which one is correct depends on the business requirement — always ask!'

8

SUM where ALL values are NULL — what is returned?

Topic: SUM + NULL

● EASY

id	amount
1	NULL
2	NULL
3	NULL

```
SELECT SUM(amount), COUNT(*), COUNT(amount) FROM Payments;
```

■ SURPRISE — Most people get this wrong!

SUM(amount) = NULL (not 0!), COUNT(*) = 3, COUNT(amount) = 0

SUM(amount)	COUNT(*)	COUNT(amount)
NULL	3	0

■ WHY THIS HAPPENS

SUM of all NULLs = NULL, not 0. There are no non-NULL values to sum. COUNT(*) = 3 because it counts all rows regardless. COUNT(amount) = 0 because no amount values are non-NULL.

■ THE FIX

Use COALESCE to get 0 instead of NULL: COALESCE(SUM(amount), 0)

SECTION 3

GROUP BY Tricks

Execution order surprises

FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY→LIMIT

9

Can you use a SELECT alias in WHERE?

Topic: Execution Order

◆ MEDIUM

```
SELECT salary * 12 AS annual
FROM Employee
WHERE annual > 500000; -- using the SELECT alias
```

■ SURPRISE — Most people get this wrong!

ERROR in all databases. WHERE runs BEFORE SELECT. The alias 'annual' does not exist yet.

■ WHY THIS HAPPENS

SQL does NOT execute in the order you write it. Actual order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. WHERE runs at step 2. SELECT runs at step 5. So aliases defined in SELECT are not yet available in WHERE.

■ THE FIX

Repeat the expression in WHERE: WHERE salary * 12 > 500000 OR wrap in a subquery/CTE: WITH t AS (SELECT salary*12 AS annual FROM Employee) SELECT * FROM t WHERE annual > 500000

Clause	Runs at step	Can use SELECT aliases?	Can use aggregates?
WHERE	2 (before GROUP BY)	NO — alias not created yet	NO
GROUP BY	3	NO	NO
HAVING	4 (after GROUP BY)	YES (some DBs)	YES
ORDER BY	7 (after SELECT)	YES	YES

10

WHERE vs HAVING with COUNT — which is valid?

Topic: WHERE vs HAVING

● EASY

```
-- Option A: WHERE with aggregate
SELECT dept, COUNT(*) FROM Employee WHERE COUNT(*) > 3 GROUP BY dept;
-- Option B: HAVING with aggregate
SELECT dept, COUNT(*) FROM Employee GROUP BY dept HAVING COUNT(*) > 3;
```

■ SURPRISE — Most people get this wrong!

Option A = ERROR. Option B = correct.

■ WHY THIS HAPPENS

WHERE runs BEFORE GROUP BY. At that point COUNT(*) does not exist yet — no groups have been formed. HAVING runs AFTER GROUP BY, so COUNT(*) is fully computed and can be filtered.

■ THE RULE — memorize this

Rule of thumb: condition involves COUNT/SUM/AVG/MIN/MAX? → Use HAVING. Condition on raw column values? → Use WHERE. You can use both in the same query.

11

HAVING without GROUP BY — is this valid? What does it return?

Topic: HAVING + GROUP BY

▲ HARD

salary

50000

60000

70000

```
SELECT AVG(salary) AS avg_salary FROM Salaries HAVING AVG(salary) > 100000;
```

■ ANSWER

Valid query — returns 0 rows. AVG = 60000, which is NOT > 100000, so the one group is filtered out.

■ WHY THIS HAPPENS

HAVING without GROUP BY treats the ENTIRE table as one single group. So this is asking: 'if the average of all salaries > 100000, show it.' The average is 60000 which fails the condition. If condition was HAVING AVG > 50000, it would return 60000.

■ THE RULE — memorize this

HAVING without GROUP BY = filter on the whole table as a single group. Valid SQL. Rarely used but can appear in interviews.

12

GROUP BY vs DISTINCT — same result?

Topic: GROUP BY vs DISTINCT

● EASY

```
SELECT DISTINCT dept FROM Employee;  
SELECT dept FROM Employee GROUP BY dept;
```

■ ANSWER

Same result — both return unique department values when no aggregates are involved.

Approach	When to use	Can add aggregates?
DISTINCT	Simple deduplication	NO
GROUP BY	Dedup + want COUNT/SUM/AVG	YES

Window Function Traps

RANK, DENSE_RANK, frames

Window functions run in SELECT — AFTER WHERE and GROUP BY

13

RANK vs DENSE_RANK vs ROW_NUMBER — exact numbers with tied salaries

Topic: Window Functions

◆ MEDIUM

name	salary
Alice	90000
Bob	90000
Carol	80000
Dave	80000
Eve	70000

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM   Employees;
```

What are the rn / rnk / drnk values for Carol?

■ ANSWER

ROW_NUMBER = 3, RANK = 3 (gap after two rank-1s), DENSE_RANK = 2 (no gap)

name	salary	ROW_NUMBER	RANK	DENSE_RANK
Alice	90000	1	1	1
Bob	90000	2	1	1
Carol	80000	3	3	2
Dave	80000	4	3	2
Eve	70000	5	5	3

Function	Output	Ties	Gap after ties?	Use for
ROW_NUMBER()	1,2,3,4,5	Always different	No gap	Dedup, pagination, top-1 per group
RANK()	1,1,3,3,5	Same rank	YES — GAP!	Sports ranking where 2nd place tie means no 3rd
DENSE_RANK()	1,1,2,2,3	Same rank	NO GAP	Top-N salary tiers — both people at rank 2 included

■ INTERVIEW GOTCHA

For 'top 3 salary tiers' → ALWAYS use DENSE_RANK. If Carol and Dave both have salary 80k at rank 3, WHERE rnk<=3 with DENSE_RANK includes both. WHERE rnk<=3 with RANK would exclude both (they are rank 3 but if two tied at rank 2, they'd be rank 4 with RANK)!

14

Can you use a window function in the WHERE clause?

Topic: Window + WHERE

▲ HARD

```
-- WRONG
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rnk
FROM   Employee
WHERE  RANK() OVER (ORDER BY salary DESC) <= 3;
-- CORRECT: wrap in CTE
WITH ranked AS (
  SELECT name, salary,
         RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM   Employee
)
SELECT name, salary FROM ranked WHERE rnk <= 3;
```

■ SURPRISE — Most people get this wrong!

ERROR! Window functions cannot be used in WHERE clause.

■ WHY THIS HAPPENS

Window functions execute in the SELECT phase (step 5). WHERE runs at step 2 — before SELECT. So at the time WHERE is evaluated, the window function result does not exist yet. Same reason you cannot use a SELECT alias in WHERE.

■ THE RULE — memorize this

Window functions CANNOT be used in: WHERE, GROUP BY, HAVING. They CAN be used in: SELECT, ORDER BY. To filter on window function results → always wrap in a CTE or subquery first.

15

LAST_VALUE — why does it return the current row, not the last row?

Topic: Window Frame

▲ HARD

Sales

What you expect vs get

date	amount	Row	Expected	Gets
Jan 1	100	Jan 1	300	100
Jan 2	200	Jan 2	300	200
Jan 3	300	Jan 3	300	300

```
-- WRONG: default frame cuts off at current row
LAST_VALUE(amount) OVER (ORDER BY date)
-- Returns: 100, 200, 300 (current row each time!)
-- CORRECT: specify full frame
LAST_VALUE(amount) OVER (
  ORDER BY date
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
-- Returns: 300, 300, 300
```

■ **SURPRISE — Most people get this wrong!**

Default frame is ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. So for row 1, the window contains only row 1 — and LAST_VALUE of that 1-row window = 100!

■ **WHY THIS HAPPENS**

When you write OVER (ORDER BY date) without a ROWS clause, SQL uses the default frame: from the beginning up to and including the CURRENT ROW. For row 1: window = [row1]. Last value = 100. For row 2: window = [row1, row2]. Last value = 200. Only for row 3 does the window contain all rows, so only row 3 gets 300.

■ **THE RULE — memorize this**

LAST_VALUE ALWAYS needs: ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.
FIRST_VALUE works fine with default frame. Only LAST_VALUE has this trap.

ORDER BY & LIMIT

Sorting surprises

16

Where do NULLs appear in ORDER BY ASC — first or last?

Topic: ORDER BY + NULL

● EASY

name	score
Alice	80
Bob	NULL
Carol	60

```
SELECT * FROM Scores ORDER BY score ASC;
```

■ SURPRISE — Most people get this wrong!

Different databases treat NULLs differently! MySQL: NULLs FIRST in ASC. PostgreSQL: NULLs LAST in ASC.

Database	ASC order	DESC order
MySQL / SQL Server	NULLs FIRST (treated as lowest)	NULLs LAST
PostgreSQL / Oracle	NULLs LAST	NULLs FIRST

■ THE FIX

Control NULL position explicitly — works in all databases: ORDER BY CASE WHEN score IS NULL THEN 1 ELSE 0 END, score ASC -- NULLs always last ORDER BY CASE WHEN score IS NULL THEN 0 ELSE 1 END, score ASC -- NULLs always first

17

LIMIT without ORDER BY — does it always return the same rows?

Topic: LIMIT + ORDER BY

◆ MEDIUM

```
-- DANGEROUS: no ORDER BY
SELECT * FROM Products LIMIT 5;
-- SAFE: always use ORDER BY before LIMIT
SELECT * FROM Products ORDER BY product_id LIMIT 5;
```

■ SURPRISE — Most people get this wrong!

Without ORDER BY, result is NON-DETERMINISTIC. Different rows can be returned each time!

■ WHY THIS HAPPENS

Relational databases do not guarantee any particular row order unless you specify ORDER BY. After inserts, deletes, index changes, or optimizer decisions — the 'first 5 rows' can change. This causes intermittent production bugs that are very hard to debug.

■ THE RULE — memorize this

ALWAYS use ORDER BY before LIMIT. No exceptions. For pagination: WHERE id > last_seen_id ORDER BY id LIMIT 10 is faster than LIMIT OFFSET on large tables.

18

Can you use ORDER BY inside UNION without parentheses?

Topic: UNION + ORDER BY

◆ MEDIUM

```
-- WRONG: ORDER BY in the middle of UNION
SELECT name FROM A ORDER BY name
UNION ALL
SELECT name FROM B;
-- CORRECT: parentheses give each SELECT its own ORDER BY + LIMIT
(SELECT name FROM A ORDER BY name LIMIT 3)
UNION ALL
(SELECT name FROM B ORDER BY name LIMIT 3);
```

■ SURPRISE — Most people get this wrong!

The first query causes an ERROR or the ORDER BY applies to the FINAL combined result, not just Table A.

■ THE RULE — memorize this

Parentheses around each SELECT allow independent ORDER BY + LIMIT BEFORE the UNION combines them. Without parentheses, ORDER BY can only appear once at the very end, sorting the final combined result.

UNION Tricks

Set operation surprises

19

UNION vs UNION ALL — how many rows from these two tables?

Topic: UNION vs UNION ALL

◆ MEDIUM

Table A

val
1
2
3

Table B

val
2
3
4

SELECT val FROM A UNION SELECT val FROM B; → How many rows?
SELECT val FROM A UNION ALL SELECT val FROM B; → How many rows?

■ ANSWER

UNION = 4 rows (deduplicates 2 and 3). UNION ALL = 6 rows (keeps all).

UNION output

val
1
2
3
4

UNION ALL output

val
1
2
3
2
3
4

■ WHY THIS HAPPENS

UNION removes duplicates — requires an internal sort/hash operation. Slower. Values 2 and 3 appear in both tables. UNION keeps one copy each (4 rows). UNION ALL keeps both copies (6 rows). UNION ALL is faster — use it when duplicates are OK.

■ THE RULE — memorize this

Use UNION ALL unless you explicitly need deduplication. UNION ALL can be 2–10x faster on large tables because it skips the dedup step.

20

UNION — which column names appear in the output?

Topic: UNION Column Names

● EASY

```
SELECT first_name AS person_name, age
FROM Customers
UNION ALL
SELECT company_name, founded_year
FROM Companies;
```

■ ANSWER

Column names come from the FIRST SELECT: 'person_name' and 'age'.

■ THE RULE — memorize this

UNION/UNION ALL rules: (1) Same number of columns in both SELECTs. (2) Compatible data types per column. (3) Column names from FIRST SELECT. (4) ORDER BY can only reference the first SELECT's column names.

UPDATE & DELETE Traps

Common mistakes that destroy data

21

Swap all M/F values — what is wrong with this two-step approach?

Topic: UPDATE CASE

◆ MEDIUM

```
-- WRONG: two-step approach destroys data!
UPDATE Salary SET sex = 'f' WHERE sex = 'm'; -- Step 1
UPDATE Salary SET sex = 'm' WHERE sex = 'f'; -- Step 2
-- CORRECT: single CASE UPDATE
UPDATE Salary SET sex = CASE WHEN sex = 'm' THEN 'f' ELSE 'm' END;
```

■ SURPRISE — Most people get this wrong!

The two-step approach sets EVERYONE to 'm'! Step 1 sets all M→F. Step 2 sets ALL F→M (including the original females!)

1	Before	M: Alice, Carol. F: Bob, Dave.
2	After Step 1	All M rows become F. Now: F: Alice, Bob, Carol, Dave.
3	After Step 2	All F rows become M. Now: M: Alice, Bob, Carol, Dave. EVERYONE IS M!

■ WHY THIS HAPPENS

CASE inside UPDATE reads all the old values FIRST, then writes new values in a single atomic operation. No row is read twice. The two-step approach has no such guarantee.

22

DELETE with subquery referencing same table — valid in MySQL?

Topic: DELETE Subquery

▲ HARD

```
-- WRONG in MySQL: can't reference same table in subquery
DELETE FROM Person WHERE id NOT IN (
  SELECT MIN(id) FROM Person GROUP BY email
);
-- ERROR: You can't specify target table 'Person' for update in FROM
-- CORRECT: wrap in derived table
DELETE FROM Person WHERE id NOT IN (
  SELECT min_id FROM (
    SELECT MIN(id) AS min_id FROM Person GROUP BY email
  ) AS tmp
);
-- OR: self-join DELETE (cleaner)
DELETE p1 FROM Person p1
JOIN Person p2 ON p1.email = p2.email AND p1.id > p2.id;
```

■ SURPRISE — Most people get this wrong!

ERROR in MySQL! MySQL disallows reading from and writing to the same table in one query.

■ WHY THIS HAPPENS

MySQL sees the subquery as trying to read from the table being deleted from — it forbids this to avoid inconsistent state. The fix: wrap the subquery in another subquery (derived table) so MySQL treats it as a separate materialized result.

■ THE RULE — memorize this

MySQL DELETE with same-table subquery: always add one extra layer of wrapping. Or use the self-join DELETE syntax which MySQL handles differently.

SECTION 8

More Gotchas

Integer division · BETWEEN · LIKE · ORDER BY aliases

23

Integer division trap — what does 5/2 return?

Topic: Integer Division

● EASY

```
SELECT 5/2; -- A
SELECT 5/2.0; -- B
SELECT COUNT(*) * 30 / 100 FROM t; -- C (if COUNT returns 3)
```

■ SURPRISE — Most people get this wrong!

PostgreSQL and SQL Server: 5/2 = 2 (truncated). MySQL: 5/2 = 2.5.

Expression	MySQL	PostgreSQL	SQL Server
5 / 2	2.5	2	2
5 / 2.0	2.5	2.5	2.5
3 / 4	0.75	0	0
COUNT(*)*30/100 (count=3)	0.9	0	0

■ WHY THIS HAPPENS

PostgreSQL and SQL Server: integer ÷ integer = integer (decimal truncated silently). 3×30 = 90. 90÷100 = 0. Your percentage query returns 0 and you get no error!

■ THE RULE — memorize this

ALWAYS use .0 for division that should be fractional: COUNT(*) * 100.0 / total. Or CAST(COUNT(*) AS DECIMAL(10,2)) / total. The .0 forces float division everywhere.

24

Is BETWEEN inclusive or exclusive on the endpoints?

Topic: BETWEEN

● EASY

event_id	event_date
1	2024-01-01
2	2024-01-15
3	2024-01-31
4	2024-02-01

```
SELECT * FROM Events WHERE event_date BETWEEN '2024-01-01' AND '2024-01-31';
```

■ ANSWER

3 rows: Jan 1, Jan 15, Jan 31. BETWEEN is INCLUSIVE on BOTH endpoints.

■ WHY THIS HAPPENS

BETWEEN x AND y is equivalent to: value >= x AND value <= y. Both endpoints included.

■ SURPRISE — Most people get this wrong!

DATETIME/TIMESTAMP trap: BETWEEN '2024-01-01' AND '2024-01-31' misses records at '2024-01-31 14:30:00'! Because '2024-01-31' = '2024-01-31 00:00:00' — only midnight is included. Fix: WHERE event_date >= '2024-01-01' AND event_date < '2024-02-01'

25

What does _ mean in LIKE — how many rows match 'Ap_le'?

Topic: LIKE Wildcards

● EASY

name
Apple
Ap_le
Ap le
Apricot
Aple
A

SELECT name FROM Products WHERE name LIKE 'Ap_le';

■ ANSWER

3 rows: Apple, Ap_le, Ap le. The underscore _ matches EXACTLY ONE character — any character.

name	Matches?	Why
Apple	YES	Ap + p + le = 5 chars – p matches the _
Ap_le	YES	Ap + _ + le = 5 chars – underscore matches the _
Ap le	YES	Ap + space + le = 5 chars – space matches the _
Apricot	NO	7 chars and doesn't end in 'le'
Aple	NO	4 chars – too short
A	NO	1 char – too short

Pattern	Meaning
LIKE 'A%'	Starts with A, any length
LIKE 'Ap_le'	Ap + exactly ONE char + le (must be 5 chars total)
LIKE '%le'	Ends with 'le', any length
LIKE '%_ %'	Contains a literal underscore (must escape with \)

26

Can you use a SELECT alias in ORDER BY?

Topic: Execution Order

● EASY

```
SELECT name, salary * 12 AS annual
FROM Employee
ORDER BY annual DESC; -- using the SELECT alias
```

■ ANSWER

Works fine! ORDER BY executes AFTER SELECT, so aliases are available.

■ WHY THIS HAPPENS

Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. ORDER BY runs at step 7 — AFTER SELECT runs at step 6. So SELECT aliases are created before ORDER BY uses them. This is DIFFERENT from WHERE (step 2) where aliases do not exist yet.

Clause	Can use SELECT alias?	Example
WHERE	NO — runs before SELECT	WHERE annual > 500000 → ERROR
GROUP BY	NO — runs before SELECT	GROUP BY annual → ERROR
HAVING	YES (some DBs)	HAVING annual > 500000 — works in MySQL
ORDER BY	YES — runs after SELECT	ORDER BY annual DESC — works everywhere

Quick Reference

All 26 Answers + Rules

One page to review before your interview

All 26 Questions — Answers at a Glance

#	Question	Answer	Key Rule
1	T1(1,1,1,2,NULL) INNER JOIN T2(1,1,1,2,4)	10 rows	$3 \times 3 + 1 = 10$. NULL=0, unmatched=0
2	LEFT JOIN same data	11 rows	+1 for T1 NULL row (appears as NULL NULL)
3	T1(1,2,3) JOIN T2(1,1,2,2,3,3)	6 rows	$1 \times 2 + 1 \times 2 + 1 \times 2 = 6$
4	CROSS JOIN T1(A,B,C) $\times$ T2(X,Y)	6 rows	No ON $\rightarrow$ every combo. $3 \times 2 = 6$
5	NOT IN with NULL in subquery	0 rows!	NULL poisons NOT IN. Use NOT EXISTS
6	email = NULL vs email IS NULL	Only IS NULL works	= NULL always returns NULL (not TRUE)
7	COUNT(*) vs COUNT(amount) with NULLs	5 vs 3	COUNT(col) skips NULLs. AVG divides by non-NULL count
8	SUM of all NULL values	NULL (not 0!)	Use COALESCE(SUM(col), 0)
9	SELECT alias in WHERE clause	ERROR	WHERE runs before SELECT. Use expression directly
10	WHERE COUNT(*) > 3 vs HAVING COUNT(*) > 3	WHERE = ERROR	Aggregates only in HAVING, not WHERE
11	HAVING without GROUP BY	Valid — 0 rows	Whole table = one group. Filter applies globally
12	GROUP BY vs DISTINCT	Same result	Without aggregates, identical. GROUP BY allows COUNT/SUM
13	Carol's RANK vs DENSE_RANK (two tied above her)	RANK=3, DENSE_RANK=2	RANK has gaps. DENSE_RANK has no gaps
14	Window function in WHERE clause	ERROR	Window runs in SELECT. Wrap in CTE first
15	LAST_VALUE without full frame	Returns current row!	Add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
16	NULLs in ORDER BY ASC (MySQL)	NULLs FIRST	MySQL: NULLs first in ASC. PostgreSQL: NULLs last in ASC
17	LIMIT without ORDER BY	Non-deterministic!	Always use ORDER BY before LIMIT
18	ORDER BY inside UNION without parentheses	ERROR	Use parentheses around each SELECT for independent ORDER BY

19	UNION vs UNION ALL row count (A+B share 2 values)	UNION=4, UNION ALL=6	UNION deduplicates. UNION ALL keeps all rows
20	Column names in UNION result	From FIRST SELECT	Always uses column names from the first SELECT
21	Two-step UPDATE to swap M/F	Sets ALL to M!	Use single CASE UPDATE — atomic, correct
22	DELETE with same-table subquery in MySQL	ERROR	Wrap in derived table or use self-join DELETE
23	5/2 in PostgreSQL	2 (not 2.5!)	Use 5/2.0 or CAST to force float division
24	BETWEEN '2024-01-01' AND '2024-01-31'	Jan 31 included	BETWEEN is inclusive on both endpoints
25	LIKE 'Ap_le' — how many rows?	3 rows	_ = exactly ONE character (any character)
26	SELECT alias in ORDER BY	Works fine	ORDER BY runs after SELECT — aliases available

These 26 questions cover every common SQL trap that appears in real interviews. The pattern: SQL does something surprising because of NULL semantics, row multiplication, execution order, or window frame defaults. If you understand WHY each answer is what it is — not just WHAT the answer is — you are ahead of 90% of SQL candidates in any interview room. ■